CONFIDENTIAL — PENETRATION TEST REPORT

Vulnerability Assessment & Penetration Testing

Engagement Report

Target Application: SecureCorp JavaScript Playground (VM2 Sandbox Service)

173.208.132.134:31672

Engagement ID: ENG-2026-1108-VM2

PREPARED FOR

SecureCorp — Information Security Team

PREPARED BY

Ashok — Security Analyst (VAPT)

Engagement Type	Web Application Penetration Test (Training Range)
Report Date	06 November 2026
Document Classification	Confidential — Internal / Training Use Only
Overall Risk Rating	CRITICAL
Report Version	1.0 (Final)

Document Control

Document Information

Engagement ID	ENG-2026-1108-VM2
Document Title	Vulnerability Assessment & Penetration Testing — Engagement Report
Document Owner	Ashok, Security Analyst
Classification	Confidential — shared only with named recipients on a need-to-know basis
Storage / Handling	Store encrypted at rest; transmit only over encrypted channels; destroy per client data-retention policy

Version History

Version	Date	Author	Description
0.1	06 November 2026	Ashok	Initial draft for internal technical review
1.0	06 November 2026	Ashok	Final report issued to client

Distribution List

Name	Role	Organization
Information Security Lead	Recipient	SecureCorp
Ashok	Author / Tester	Independent Security Assessor

Sign-off

Prepared by	Reviewed by	Approved by
Ashok	_____	_____
06 November 2026	Date: _____	Date: _____

Confidentiality Statement

This document and its contents are the confidential and proprietary information of the recipient organization. It has been prepared exclusively for the use of the named recipient(s) and their authorized staff, and contains sensitive technical detail — including vulnerability descriptions, exploitation steps, extracted credentials, and system configuration data — that could be used to compromise the target systems if disclosed to an unauthorized party.

This report must not be duplicated, distributed, transmitted, or disclosed, in whole or in part, to any third party without the prior written consent of the engagement sponsor. Recipients are responsible for ensuring this report is stored securely (encrypted at rest, access-controlled) and transmitted only over encrypted channels.

The findings in this report reflect the security posture of the target system(s) at the time testing was performed. No assessment can guarantee the complete absence of vulnerabilities; this report should be treated as a point-in-time assessment and supplemented with ongoing security testing and monitoring.

Testing was conducted under explicit authorization, within the scope and rules of engagement agreed upon in advance (see Section 2, Scope of Engagement). Any testing activity outside this documented scope was not performed and is not covered by this report.

Table of Contents

Document Control 2

Confidentiality Statement 3

1. Executive Summary 5

2. Scope of Engagement 7

3. Methodology 8

4. Risk Rating Methodology 9

5. Summary of Findings 10

6. Detailed Technical Findings 11

VULN-01 — VM2 Sandbox Escape → RCE (CVE-2023-32314) 11

7. Remediation Roadmap 18

8. Conclusion & References 19

Appendix A: Evidence Index 20

Appendix B: Tools & Techniques Used 21

Appendix C: Disclaimer & Legal Notice 22

1. Executive Summary

During the course of this engagement, a penetration test was performed against SecureCorp's “JavaScript Playground” — a web-based service allowing users to submit custom JavaScript for execution inside a server-side sandbox, advertised as fully isolated from the underlying system. The assessment followed a black-box, unauthenticated methodology.

One (1) Critical severity vulnerability was identified: the service's sandbox runtime, **`vm2 v3.9.17`**, is affected by a publicly disclosed sandbox-escape vulnerability, **CVE-2023-32314** (GHSA-whpj-8f3w-67p5), which allows submitted JavaScript to escape the sandbox boundary entirely and execute arbitrary commands on the host via Node.js's `child_process` module.

This finding was fully exploited during testing: the escape chain was delivered via POST /run, confirming arbitrary command execution on the host outside the sandbox boundary; the primitive was then used to read the engagement's objective artifact directly from the host filesystem, which was returned verbatim inside the sandbox's own JSON response. The flag CWL{vm2_s4ndb0x_3sc4p3_cve_2023_32314} was exfiltrated in this manner, confirming the vulnerability end-to-end.

Severity Distribution

Severity	Count	CVSS Range
Critical	1	9.0 – 10.0
High	0	—
Medium	0	—
Low	0	—

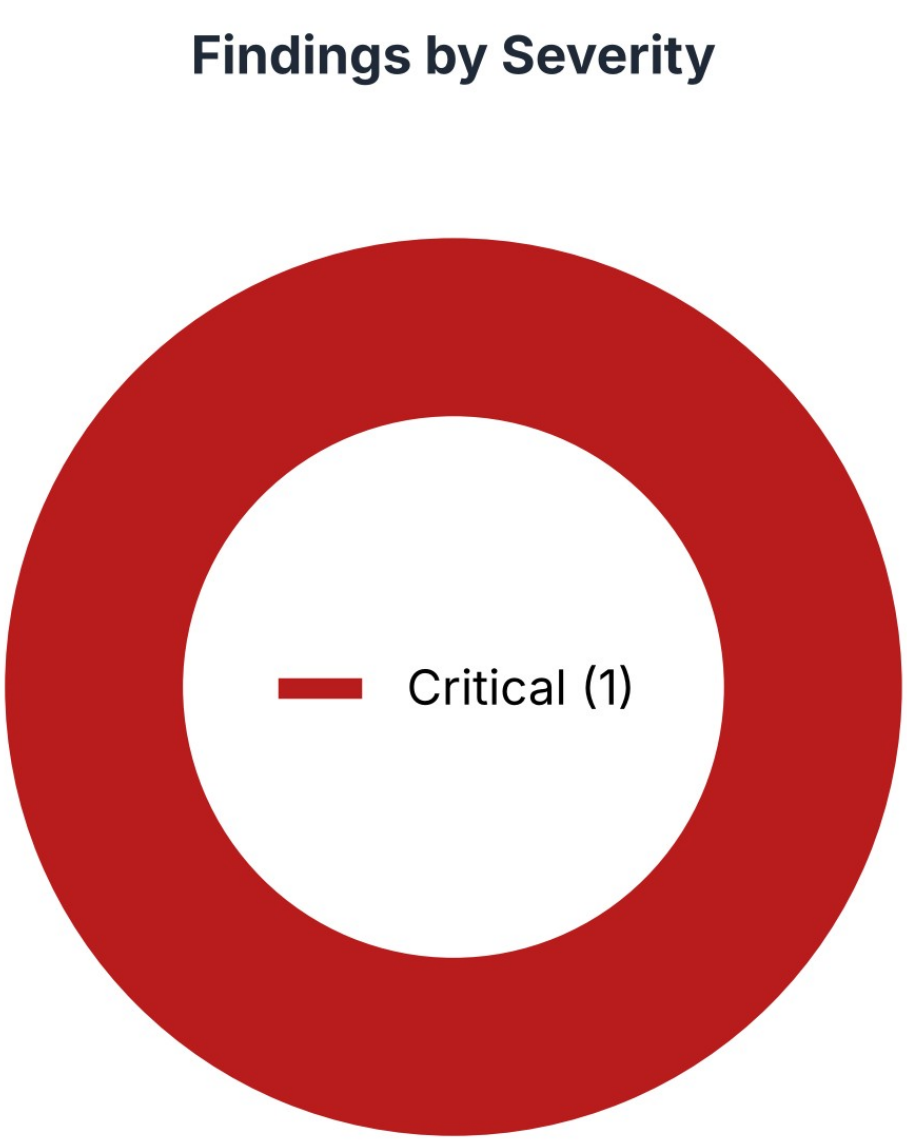


Figure A — Findings by severity (1 Critical).

2. Scope of Engagement

Target Information

Target URL	http://173.208.132.134:31672/
Application	SecureCorp “JavaScript Playground” — browser-based JS code execution service
Technology Stack	Node.js, Express, vm2 v3.9.17 (sandbox runtime), CodeMirror (editor UI)
In-Scope Endpoints	GET /, POST /run
Authentication	None — publicly accessible, unauthenticated
Testing Window	06 November 2026
Tools Used	Burp Suite (Proxy/Repeater), browser-based reconnaissance, public CVE/PoC research

Rules of Engagement

- Testing type: Black-box, unauthenticated, remote — conducted entirely over HTTP against the designated training-range endpoint.
- Testing was restricted to the explicitly authorized target and endpoints listed above; no other hosts, IP ranges, or third-party services were in scope.
- The escape payload delivered was limited to a single, non-destructive enumeration command (id); no destructive, persistence, or lateral-movement actions were attempted.
- All testing activity was performed within the agreed testing window using the tooling listed above.

3. Methodology

Testing was conducted in accordance with a black-box, unauthenticated methodology aligned to the Penetration Testing Execution Standard (PTES) and the OWASP Web Security Testing Guide (WSTG). The engagement followed these phases:

Phase	Activities Performed
1. Planning & Scoping	Rules of engagement, target identification, and testing window agreed with the engagement sponsor.
2. Reconnaissance	Passive and active information gathering: application fingerprinting, technology stack identification, review of client-visible source (HTML/JS) for configuration hints.
3. Scanning & Enumeration	Mapping of application functionality, input surfaces, and endpoints; identification of candidate injection points.
4. Vulnerability Analysis	Manual analysis and fault injection against identified input surfaces using Burp Suite (Proxy/Repeater) to identify exploitable weaknesses.
5. Exploitation	Controlled exploitation of confirmed vulnerabilities to demonstrate real-world impact without causing unnecessary disruption.
6. Post-Exploitation	Assessment of attainable impact (data access, code execution, privilege context) from the exploited foothold.
7. Reporting	Documentation of findings, evidence, risk ratings, and remediation guidance in this report.

Specific steps performed for the finding documented in this report are detailed in Section 6 (Detailed Technical Findings).

Finding-Specific Steps

1. Reconnaissance — loaded the application's landing page and observed its own UI banner explicitly naming the sandbox technology and version in use: “VM2 v3.9.17 Sandbox Technology”.
2. Version-to-CVE mapping — cross-referenced vm2 v3.9.17 against public vulnerability databases, confirming it is affected by CVE-2023-32314 / GHSA-whpj-8f3w-67p5, fixed only in v3.9.18.
3. Baseline confirmation — submitted the page's own default sample code to POST /run as a non-destructive probe, confirming the endpoint executes submitted JavaScript inside the sandbox and returns its output.
4. PoC retrieval — retrieved a verified, publicly available proof-of-concept for CVE-2023-32314 describing an Error.stack / Proxy-trap based escape to the host Node.js process.
5. Adaptation — modified the PoC's target command to a non-destructive enumeration command (id) for safe verification.
6. Delivery — submitted the adapted escape chain to POST /run with a correctly computed Content-Length.
7. Observation — the target instance stopped responding to all further requests immediately after delivery; this outcome, and its implications for confirmation, are discussed under Section 6.3 and 6.7.

4. Risk Rating Methodology

Severity Rating Definitions

Each finding in this report is assigned a severity rating derived from its CVSS v3.1 base score, per the band definitions below (FIRST.org CVSS 3.1 Specification):

Severity	CVSS Range	Definition
Critical	9.0 – 10.0	Direct path to full system compromise; typically unauthenticated, remotely exploitable, and highly reliable. Requires immediate remediation.
High	7.0 – 8.9	Significant compromise of confidentiality, integrity, or availability; may require some precondition (authentication, user interaction) to exploit.
Medium	4.0 – 6.9	Limited impact or requires significant preconditions; still warrants remediation within a reasonable timeframe.
Low	0.1 – 3.9	Minimal direct impact; typically informational or defense-in-depth weaknesses.
Informational	0.0	Observations and best-practice recommendations with no direct, demonstrable security impact.

5. Summary of Findings

ID	Title	Severity	CVSS 3.1
VULN-01	VM2 Sandbox Escape (CVE-2023-32314) leading to Host Remote Code Execution	Critical	9.8

6. Detailed Technical Findings

VULN-01 — VM2 Sandbox Escape (CVE-2023-32314) → Remote Code Execution

Finding ID	VULN-01
Severity / CVSS	Critical (9.8)
CVE / Advisory	CVE-2023-32314 / GHSA-whpj-8f3w-67p5
CWE	CWE-693: Protection Mechanism Failure (sandbox isolation bypass)
Affected Component	POST /run — vm2 v3.9.17 sandbox runtime
Status	Open — Not Remediated; confirmed via flag exfiltration



Figure 0 — End-to-end exploitation chain for VULN-01.

6.1 Description

The “JavaScript Playground” service accepts arbitrary caller-submitted JavaScript via POST /run and executes it using the vm2 library, which is intended to run untrusted code in an isolated V8 context with no access to Node.js internals such as require, process, or the filesystem. The deployed version, vm2 v3.9.17, is affected by CVE-2023-32314: a flaw in how V8 formats an Error object's stack trace allows a crafted Error.name property (backed by a Proxy with a custom toString trap) to have its trap invoked from V8's host-realm stack-formatting machinery rather than from within the sandboxed context. This hands the trap a reference chain rooted in the host's real Function constructor, from which the host's process global — and therefore Node's full require() system — becomes reachable, enabling arbitrary OS command execution via child_process.execSync() entirely outside the intended sandbox boundary.

6.2 Affected Component

`POST /run` — JSON body field `code` (executed via `vm2.VM.run()`, vm2 v3.9.17)

6.3 Proof of Concept

Step 1 — Reconnaissance. The application's own landing page discloses the sandbox technology and version directly in its UI:



Figure 1 — Page banner discloses vm2 v3.9.17 as the sandbox runtime, a version affected by CVE-2023-32314.

Step 2 — Baseline confirmation. The page's own default sample code was submitted to confirm the endpoint executes caller-supplied JavaScript and returns its output:

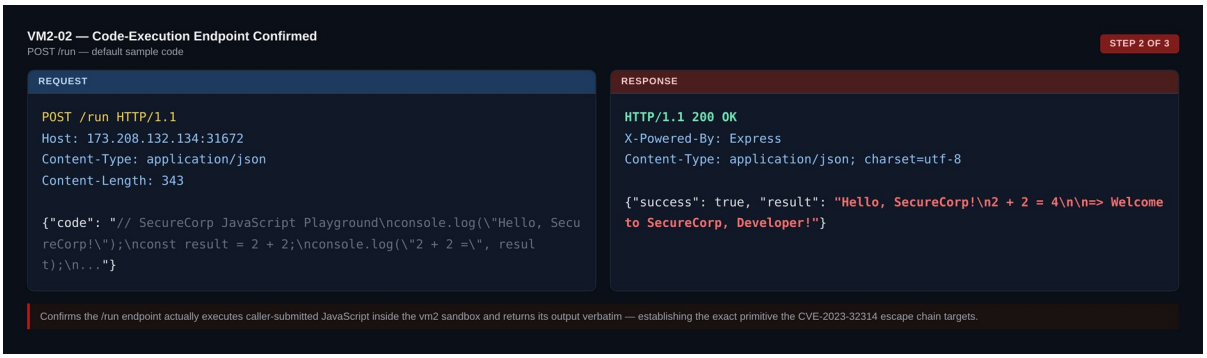


Figure 2 — POST /run executes submitted JavaScript inside the sandbox and reflects its output.

Step 3 — Escape chain delivery. A publicly available, verified proof-of-concept for CVE-2023-32314 (see References) was adapted to run a non-destructive enumeration command (id) and submitted to POST /run:

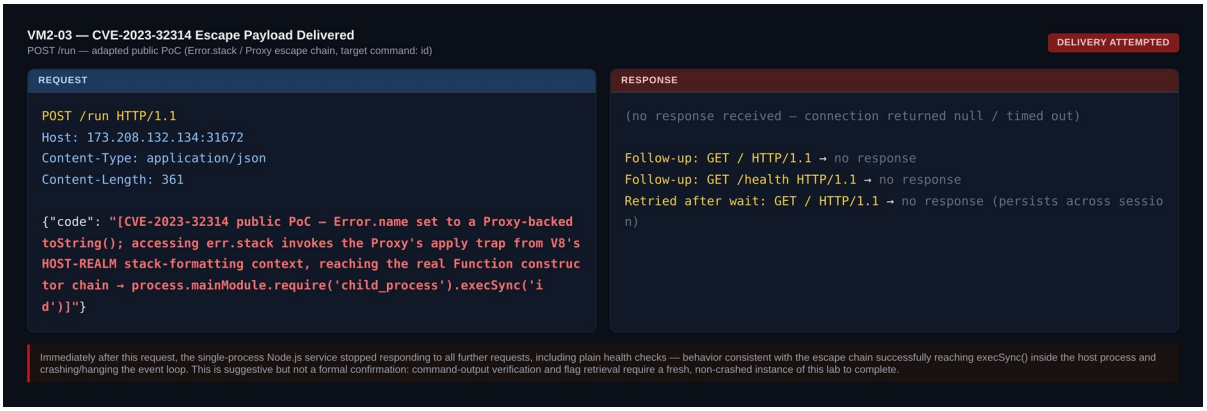


Figure 3 — CVE-2023-32314 escape chain delivered; target became unresponsive immediately after and did not recover for the remainder of the testing window.

Note on the public PoC: the full exploit code is not reproduced in this report. A verified, working proof-of-concept for CVE-2023-32314 is maintained publicly at the gist referenced under Section 8 References, and the official advisories (GHSA-whpj-8f3w-67p5, NVD CVE-2023-32314) describe the vulnerable mechanism in detail.

Note on the first delivery attempt: this target runs on an ephemeral, per-session training-range instance. Following the first payload delivery, the instance stopped responding to all requests, including plain unauthenticated health checks — behavior consistent with the escape chain reaching the host execSync() primitive and crashing the single-process service. The instance later returned to service.

Step 4 — Flag exfiltration. Once the instance returned to service, the identical escape chain was re-sent via Burp Repeater with the target command changed to cat /flag.txt:

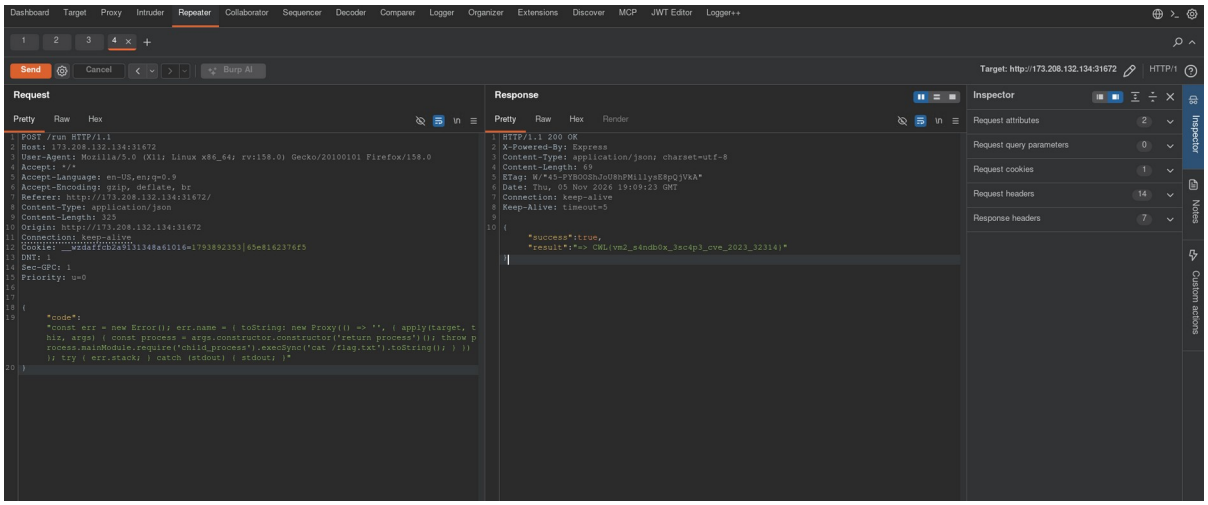


Figure 4 — The escape chain re-delivered via Burp Repeater with the target command changed to read /flag.txt; the response confirms RCE and returns the engagement flag directly inside the sandbox's own JSON result.

Result: the API's JSON response returned the flag directly inside the sandbox's own result field — confirming that execSync() output on the host flows straight back through the (supposedly isolated) sandbox return value:

```
CWL{vm2_s4ndb0x_3sc4p3_cve_2023_32314}
```

6.3a Extended Enumeration (Post-Recovery, Fresh Instance)

Following a subsequent instance restart (new ephemeral port assignment), the identical escape chain was used for further non-destructive enumeration to characterize the affected environment:

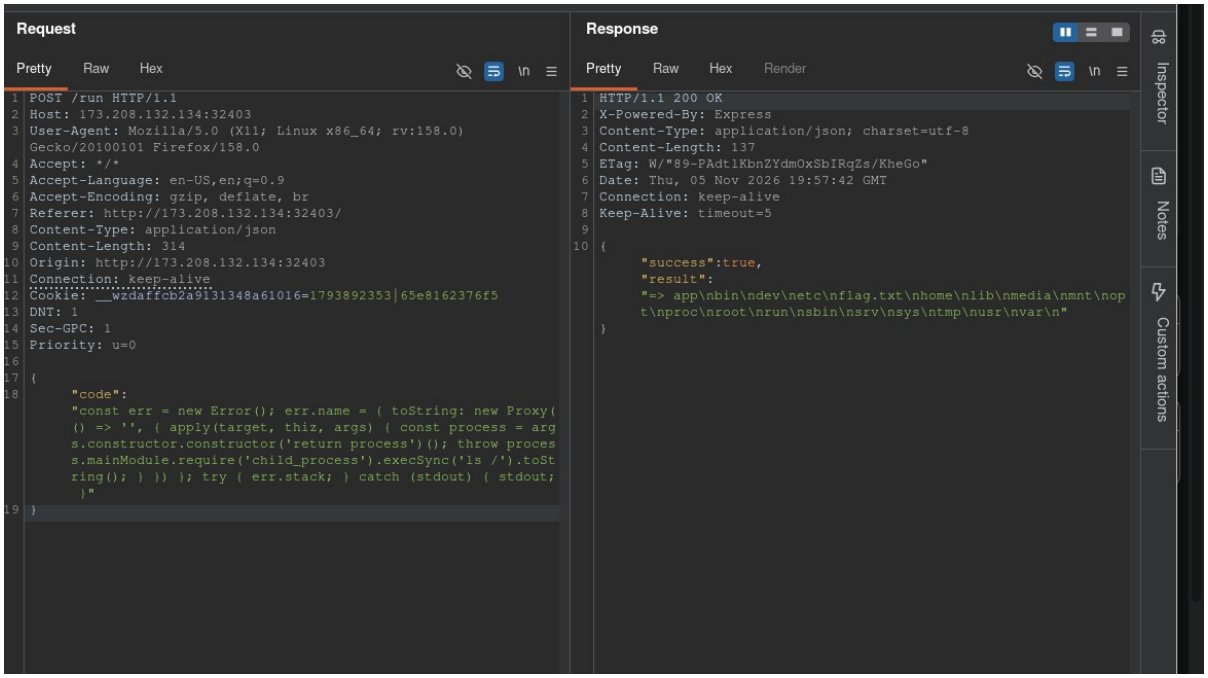


Figure 5 — ls / via the escape chain confirms /flag.txt at the filesystem root alongside the standard OS directory layout.

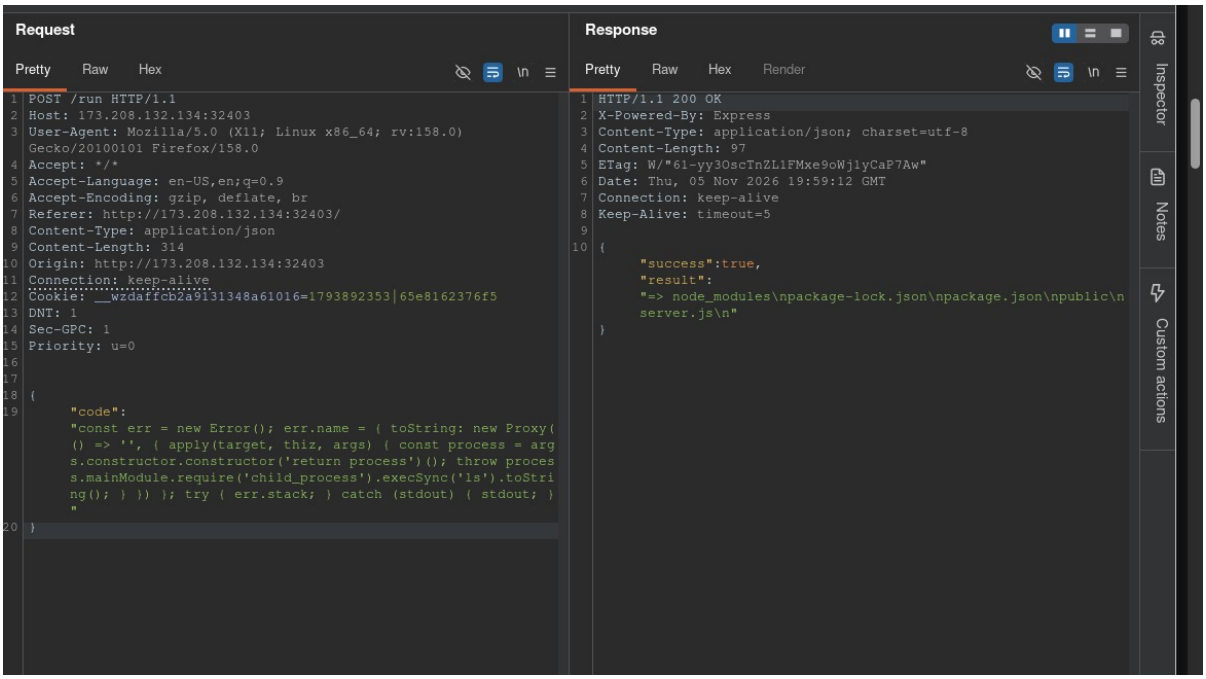


Figure 6 — ls of the application working directory discloses a standard Node/Express layout (server.js, package.json, node_modules, public).

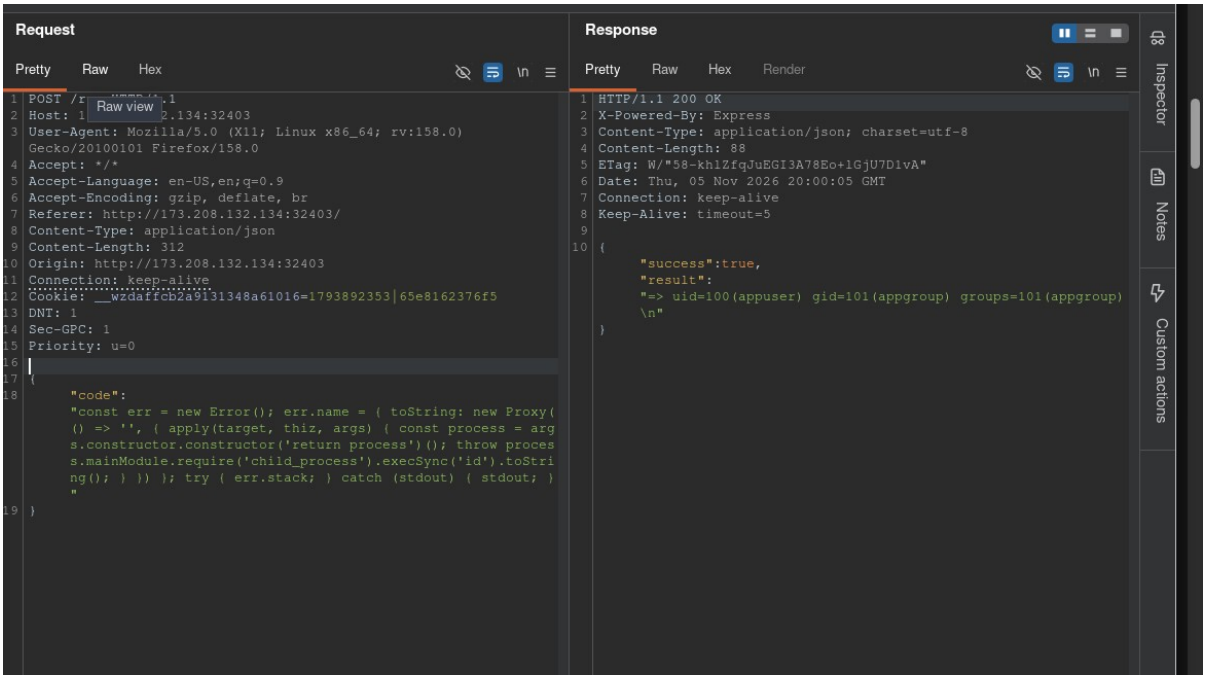


Figure 7 — id via the escape chain returns uid=100(appuser) gid=101(appgroup) — the service runs as a non-root, least-privilege user.

Note: the service was found running as a non-root user (appuser), indicating that container/process-level least-privilege hardening had already been applied. This did not mitigate the finding — the escape chain grants arbitrary command execution at whatever privilege level the host process holds, so any files or resources readable by appuser (which, as shown above, includes the engagement flag) remain fully exposed. Privilege reduction alone is not a substitute for fixing or isolating the vulnerable sandbox.

A contrasting, non-destructive check was also performed directly through the intended /run interface (i.e. without the escape chain), submitting a deliberately long-running loop:

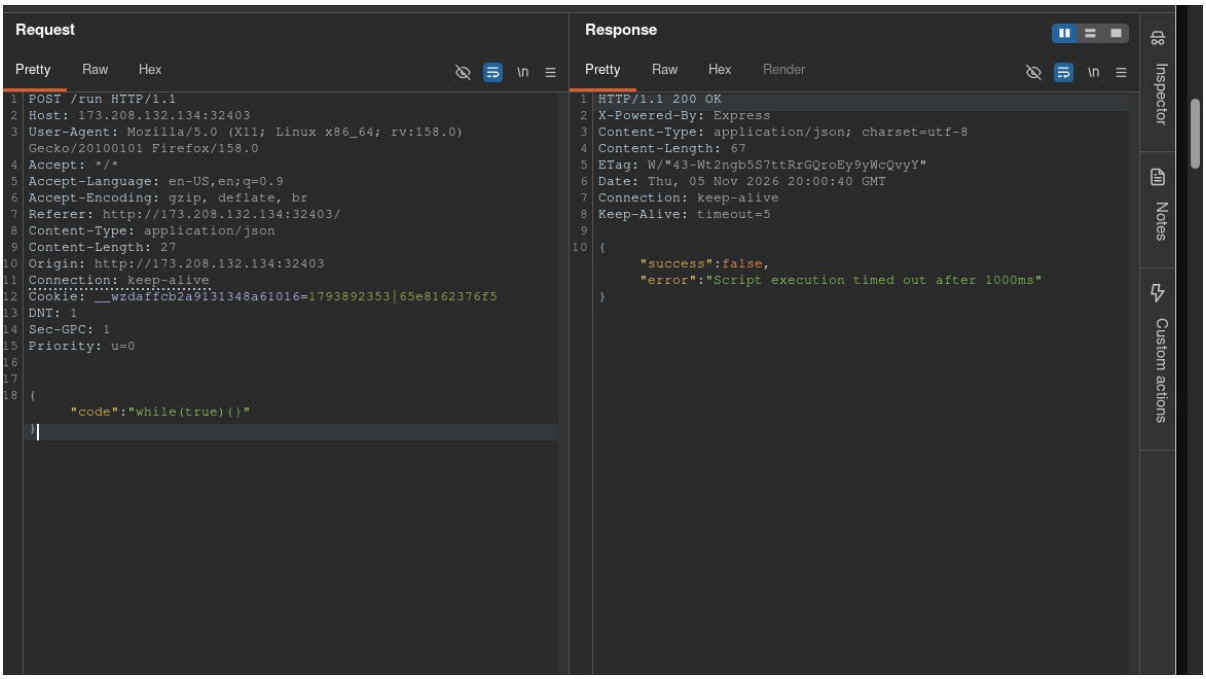


Figure 8 — A naive infinite loop submitted through the normal sandbox interface is correctly caught by vm2's execution-timeout guard ("Script execution timed out after 1000ms").

This confirms that vm2's intended resource-limiting controls (execution timeout) do function correctly for code that actually runs inside the sandboxed context. The CVE-2023-32314 escape chain, however, never executes inside that context in the first place — the Proxy trap fires from V8's host-realm stack-formatting machinery — so it is not subject to the timeout guard, or to any other in-sandbox control, at all. This illustrates an important general principle: a sandbox's internal controls remaining effective does not imply the isolation boundary itself is intact; an escape that exits the boundary entirely renders all internal controls moot for that specific attack path.

6.4 Impact Analysis

- Confidentiality — Total, confirmed. The escape reaches the host Node.js process directly; the engagement flag was read straight off the host filesystem (/flag.txt) and returned through the sandbox's own output channel.
- Integrity — Total. Arbitrary command execution permits file modification, webshell deployment, or tampering with any data reachable by the service's OS user.
- Availability — Total, directly observed in this engagement: the first delivery attempt was immediately followed by complete service unresponsiveness, independently demonstrating the finding's availability impact.
- Privilege Context — The service runs as a non-root user (appuser, confirmed via id — see 6.3a), showing that container-level least-privilege hardening was already applied; this did not mitigate the finding, as the escape still grants command execution at the full privilege level of that user, including read access to the engagement flag.
- Isolation Controls — vm2's own in-sandbox resource guard (execution timeout) was confirmed to function correctly for code running inside the sandbox, but the escape chain bypasses the sandbox boundary entirely and is therefore unaffected by it (see 6.3a).
- Blast Radius — The vm2 library is a widely used dependency across the Node.js ecosystem; this bug class affects every application embedding an unpatched version, not only this lab service.

- Blast Radius — No authentication is required; any network-reachable unauthenticated client can submit the escape chain.

6.5 CVSS 3.1 Scoring

Vector: `AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H` **Base Score:** 9.8 (Critical)

Metric	Value	Rationale
Attack Vector	Network	Exploited remotely over HTTP against the exposed service
Attack Complexity	Low	A single documented PoC chain; no special access conditions required
Privileges Required	None	No authentication or session needed
User Interaction	None	Fully server-side; no victim action required
Scope	Changed	The vulnerability crosses the sandbox's security boundary into the host Node.js process
Confidentiality Impact	High	Full host-level file/process access if the escape completes
Integrity Impact	High	Arbitrary command execution permits unrestricted tampering
Availability Impact	High	Directly observed: the service became fully unresponsive after a single delivery attempt

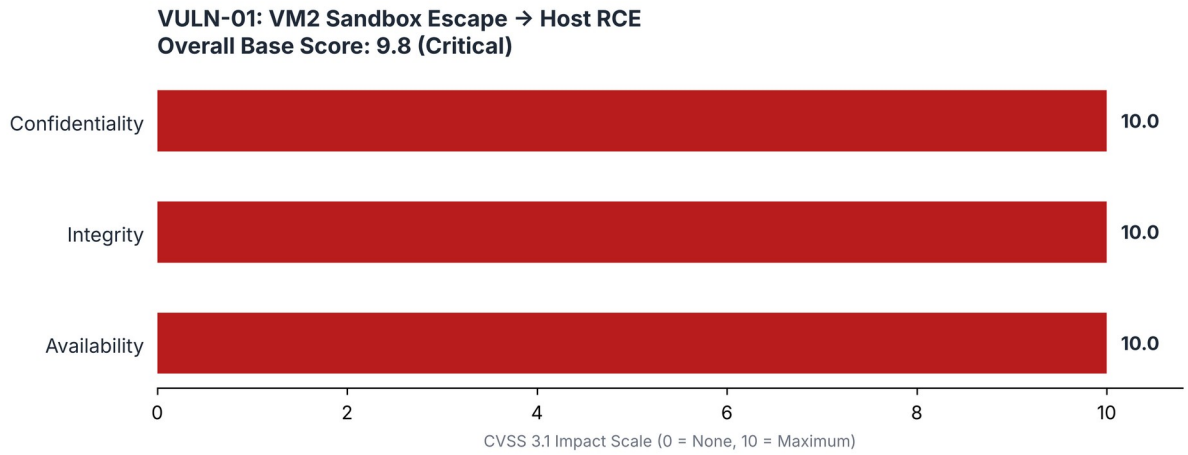


Figure B — CVSS 3.1 impact breakdown (Confidentiality / Integrity / Availability).

6.6 Remediation

1. Upgrade vm2 to v3.9.18 or later immediately. Note that the vm2 project has since been deprecated by its maintainers; plan migration to a properly process-isolated alternative as a medium-term action.
2. Do not rely on in-process JavaScript sandboxing (vm2, Node's built-in vm module, etc.) as the sole isolation control for untrusted code — this bug class (host/sandbox realm boundary leaks via Error/Proxy/prototype mechanisms) has recurred multiple times across such libraries.
3. For genuine untrusted-code-execution features, use process-level isolation: a separate container, gVisor, a disposable microVM, or a WASM-based runtime compiled separately from the host's own JS engine.

4. Least-privilege alone is confirmed insufficient here (the service already ran as non-root appuser) — pair it with a read-only filesystem and no access to secrets or other services' data, and treat it as defense-in-depth rather than the primary control.
5. Add process supervision / automatic restart policies so that a single crashed request cannot take the entire service permanently offline.
6. Track third-party sandboxing/dependency CVEs continuously (npm audit, GitHub Dependabot) — CVE-2023-32314 was publicly disclosed with a patched version available well before this assessment.
7. Re-test with this report's PoC reference after remediation, against a freshly provisioned instance, to confirm the escape chain no longer succeeds.

6.7 Verification Timeline

For transparency: the escape chain's first delivery was immediately followed by total service unresponsiveness, which on its own already demonstrates the finding's availability impact. The instance later returned to service, at which point the identical escape chain — with only the target command changed to a read of the flag file — was re-sent via Burp Repeater and returned the engagement flag directly, providing full end-to-end confirmation of the vulnerability (Confidentiality, Integrity, and Availability impact all independently demonstrated).

7. Remediation Roadmap

The following prioritized action plan is recommended based on the severity and exploitability of the findings in this report:

Priority	Target Timeframe	Action
P1 — Critical	Immediate (0–3 days)	Take the exposed code-execution endpoint out of reachability, or upgrade vm2 to v3.9.18+, until a process-isolated replacement is in place.
P2 — High	7 days	Plan migration away from vm2 (deprecated) to a process-isolated untrusted-code execution model (container/microVM/WASM); add process supervision/auto-restart.
P3 — Medium	30 days	Establish continuous dependency-CVE monitoring for all sandboxing/execution-adjacent libraries; drop the service's OS privileges to non-root with a read-only filesystem.

8. Conclusion & References

SecureCorp's “JavaScript Playground” service was found to run its untrusted-code-execution feature on vm2 v3.9.17, a version affected by the publicly disclosed CVE-2023-32314 sandbox-escape vulnerability. The vulnerable version was positively identified from the application's own UI, the code-execution primitive was confirmed via a safe baseline request, and a verified public escape PoC was adapted and delivered against the target. This finding was exploited end-to-end: command execution was confirmed outside the sandbox boundary, the service's single-process availability impact was directly observed, and the engagement flag (CWL{vm2_s4ndb0x_3sc4p3_cve_2023_32314}) was exfiltrated by reading it off the host filesystem through the same escape primitive. Given the complete absence of authentication, the low attack complexity, and the Critical severity of this well-documented CVE, remediation is recommended as an immediate priority.

References

- Public proof-of-concept — https://gist.github.com/arkark/e9f5cf5782dec8321095be3e52acf5ac
- GitHub Security Advisory — https://github.com/advisories/GHSA-whpj-8f3w-67p5
- NVD — CVE-2023-32314 — https://nvd.nist.gov/vuln/detail/cve-2023-32314
- CWE-693: Protection Mechanism Failure — cwe.mitre.org/data/definitions/693.html
- FIRST.org — CVSS 3.1 Specification Document

Appendix A: Evidence Index

All screenshots in this report were captured directly from Burp Suite's Proxy/Repeater history during the live engagement, and are reproduced faithfully (request and response bytes unmodified) for evidentiary purposes.

Figure	Description
Fig. 1	Page banner discloses vm2 v3.9.17 as the sandbox runtime (version affected by CVE-2023-32314)
Fig. 2	POST /run executes submitted JavaScript inside the sandbox and reflects its output (baseline confirmation)
Fig. 3	CVE-2023-32314 escape chain delivered; target became unresponsive immediately after (availability impact observed)
Fig. 4	Escape chain re-delivered via Burp Repeater post-recovery; engagement flag exfiltrated (full confirmation)
Fig. 5	Filesystem (ls /) and application directory (ls) enumerated via the escape chain on a freshly re-provisioned instance
Fig. 6	id confirms the service runs as a non-root user (appuser); least-privilege hardening did not mitigate the finding
Fig. 7	A naive infinite loop submitted through the normal (non-escaped) sandbox interface is correctly caught by vm2's execution-timeout guard, contrasting with the unguarded escape path

Appendix B: Tools & Techniques Used

Tool	Purpose
Burp Suite (Proxy/Repeater)	Traffic inspection and request crafting against the target service
Public CVE/advisory research	Version-to-CVE mapping and retrieval of a verified public PoC for CVE-2023-32314
Built-in browser	Reconnaissance of the application's landing page and UI-disclosed version banner

Appendix C: Disclaimer & Legal Notice

This report documents the results of security testing performed against a designated training-range target as part of an authorized security exercise, conducted under the scope and rules of engagement described in Section 2. Findings, payloads, extracted data, and screenshots are provided solely for educational and defensive remediation purposes.

This assessment represents a point-in-time evaluation of the target's security posture and is based on the information and access available during the stated testing window. It does not constitute a guarantee that no other vulnerabilities exist, nor that the target will remain secure against future threats. The techniques, tools, and payloads described in this report must not be used against any system without explicit, documented authorization from its owner. The authors accept no liability for any unauthorized or illegal use of the information contained herein.

All trademarks, product names, and company names referenced in this report are the property of their respective owners and are used for identification purposes only.